

[_3D.SynchronizedJoints \[SimTalk\]](#)

Sets if moving toward a pose will synchronize the joints, so that all joints, that are part of the movement, reach the target state at the same time (`true`) or not (`false`).

Type

Attribute

Syntax

```
<Path>._3D.SynchronizedJoints:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
.MaterialFlow.Station._3D.SynchronizedJoints := true
```

SimTalk

[__Accessing Poses](#)

See also

[Poses](#)

[Synchronized Joints](#)

Accessing the Worker

[__Accessing the Worker](#)

SimTalk provides the *attributes* listed in the table of contents to the left for the **Graphic Sequences** of the *Worker*.

See also

[Edit 3D Properties \[dialog\]](#) > [Graphic Sequences](#)

[_3D.GraphicsWhenCarrying \[SimTalk\]](#)

Sets a sequence of graphic groups of the *Worker* designated by <Path>, while he carries a part.

Remarks

The *Worker* cycles through the graphic groups in the sequence which you enter here, when he walks on a *FootPath*, or he walks freely within the model, while he carries a part.

If this sequence is empty, 3D uses the visible graphic groups as defined in the attribute *_3D.VisibleGraphics*, i. e., the default behavior.

Type

Attribute

Syntax

```
<Path>._3D.GraphicsWhenCarrying:string[]
```

Assignment Value

You can assign an *array* of data type *string*.

Example

```
.Resources.MyWorker._3D.GraphicsWhenCarrying := ["Pose1", "Pose2"]
```

See also

[Edit 3D Properties \[dialog\]](#) > [Graphic Sequences](#) > [Walk and Carry Part](#)

[_3D.VisibleGraphicGroups \[SimTalk\]](#)

[_3D.VisibleWalkingGraphicGroup \[SimTalk\]](#)

[_3D.GraphicsWhenEmpty \[SimTalk\]](#)

Sets a sequence of graphic groups of the *Worker* designated by <Path>, while he does not carry a part..

Remarks

The *Worker* cycles through the graphic groups in the sequence which you enter here, when he walks on a *FootPath*, or when he freely within the model, while he does not carry a part.

If this sequence is empty, *Plant Simulation* uses the visible graphic groups as defined in the attribute *_3D.VisibleGraphics*, i. e., the default behavior.

Type

Attribute

Syntax

```
<Path>._3D.GraphicsWhenEmpty:string[]
```

Assignment Value

You can assign an *array* of data type *string*.

Example

```
.Resources.MyWorker._3D.GraphicsWhenEmpty := ["Pose1", "Pose2"]
```

See also

[Edit 3D Properties \[dialog\]](#) > [Graphic Sequences](#) > [Walk Without Part](#)

[_3D.VisibleGraphicGroups \[SimTalk\]](#)

[_3D.VisibleWalkingGraphicGroup \[SimTalk\]](#)

_3D.ObstacleForWorker [SimTalk]

Sets which part of the object designated by <Path> is an obstacle for the *Worker* when he is walking freely within the area of the model.

Remarks

To show the **Barred Areas/Obstacles** for the *Worker* in your model, press the **o** key on your keyboard or



click on the tab **View** of the *Frame* window.

Type

Attribute

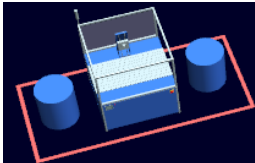
Syntax

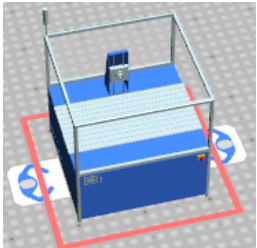
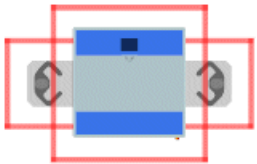
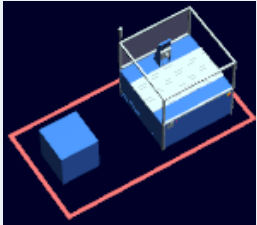
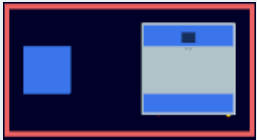
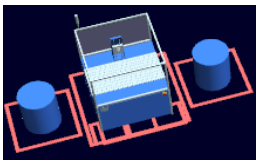
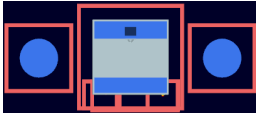
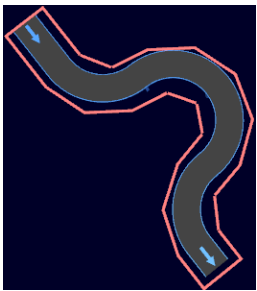
```
<Path>._3D.ObstacleForWorker:string
```

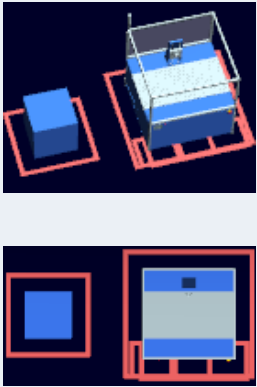
Assignment Value

You can assign a value of data type *string*.

You can specify one of the following settings:

Setting	Description	Looks like this
"Bounding box"	The bounding box of the entire object is an Obstacle for the Worker around which he has to walk. This applies for example for the <i>material flow objects</i>, such as the <i>Station</i>, etc., and for graphics. For the <i>Workplace Plant Simulation</i> shows two bounding boxes with a gap in between. This ensures that the <i>Worker</i> only stays on the middle of the <i>Workplace</i>. The <i>Worker</i> first has to leave the <i>Workplace</i> before he can change his walking direction forward the next destination object. Bounding box is the most performant setting.	 

Setting	Description	Looks like this
		 
"Bounding box of visible graphics"	The bounding box of the entire object is an Obstacle for the Worker around which he has to walk. Graphic groups, which are hidden at the moment, are not considered. <i>Length-oriented objects and Frames do not provide this setting.</i>	 
"Graphics"	The individual graphics of the object are an Obstacle for the Worker around which he has to walk. In the example to the right the <i>Worker</i> can walk through the pillars and the <i>Station</i> without having to walk around the overall obstacle. This applies for example for the <i>length-oriented material flow objects</i>, such as the <i>Conveyor</i>. For the object <i>Conveyor Plant Simulation</i> approximates the display of the bounding box only roughly to the curve, i.e., it does not show the curve of the conveyor as a smooth curved line. The setting Graphics can have negative effects on the performance when you switch the display often.	  

Setting	Description	Looks like this
"Visible graphics"	The graphics of the object are an Obstacle for the Worker around which he has to walk. Graphic groups, which are hidden at the moment, are not considered. This applies for example for the <i>length-oriented material flow objects</i>, such as the <i>Conveyor</i>. The setting Visible graphics can have negative effects on the performance when you switch the display often.	
"(None)"	The object is no obstacle for the <i>Worker</i> . He can walk through the object or across it. This applies for example for the <i>information flow objects</i> , such as the <i>Method</i> , etc.	

Examples

```
Station._3D.ObstacleForWorker := "Graphics"
```

```
PickAndPlace._3D.ObstacleForWorker := "(None)"
```

SimTalk

_3D.getGraphic.[ObstacleForWorker \[SimTalk\]](#) of a graphic

See also

[Edit 3D Properties \[dialog\]](#) > [Graphics \[tab\]](#) > [Obstacle for the Worker \[drop-down list\]](#)

[Edit 3D Properties \[dialog\]](#) > [Graphic Settings](#) > [Obstacle for the Worker](#)

Show Obstacles

[Worker Width \[model settings\]/Worker Width \[preferences\]](#)

Barred Area

[Model a Worker Who Walks Freely Within the Model](#)

[Worker \[object\]](#)

WorkerPool > **Travel Mode** [drop-down list]

[_3D.VisibleWalkingGraphicGroup \[SimTalk\]](#)

Returns the name of the graphic group that the *Worker* instance designated by <Path> is currently using to visualize a walking *Worker*.

Remarks

If the *Worker* is currently not walking, *_3D.VisibleWalkingGraphicGroup* returns an empty string "".

Type

Read-only Attribute

Syntax

```
<Path>._3D.VisibleWalkingGraphicGroup:string
```

Assignment Value

You can assign a value of data type *string*.

Example

```
print .Resources.Worker:1._3D.VisibleWalkingGraphicGroup  
-- might, for example, return carrying0
```

See also

[Edit 3D Properties \[dialog\]](#) > [Graphic Sequences](#)

[_3D.GraphicsWhenCarrying \[SimTalk\]](#)

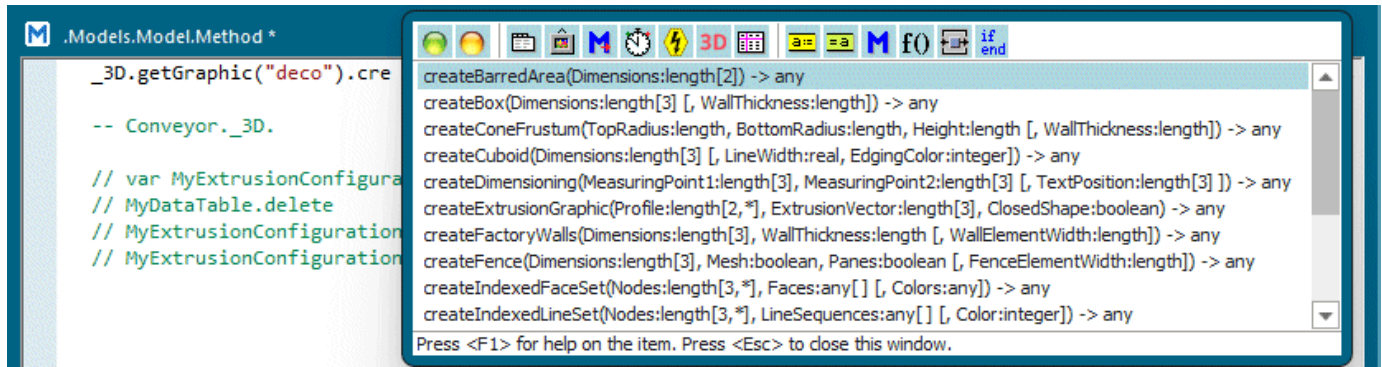
[_3D.GraphicsWhenEmpty \[SimTalk\]](#)

Accessing Methods of Graphic Shapes

[_Accessing Methods of Graphic Shapes](#)

SimTalk provides the *methods* listed in the table of contents to the left for creating **shapes**.

You can view and access the **methods** for accessing graphics by clicking **Auto Complete** on the **Edit** ribbon tab of the *Method Editor*.



You can also insert shapes with the UI by clicking **Insert Shape** on the **Edit** ribbon tab.

See also

[Auto Complete](#)

[Insert Shape](#)

[_Accessing Attributes of Graphic Shapes](#)

createBarredArea [SimTalk]

Creates a barred area in the graphic or graphic group designated by <Path>.

Remarks

Initially the created graphic does not have any materials. You can set these later.

Type

Method

Syntax

```
<Path>.createBarredArea(Dimensions:length[2]) → any
```

Parameter

The parameter *Dimensions* is an *array* of data type *length* which sets the **Width/dimension X** and the **Depth/dimension Y** of the barred area.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Examples

```
var BarredArea := _3D.getGraphic("deco").createBarredArea([8, 6])
BarredArea.MaterialActive := true
BarredArea.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

```
var BarredArea := _3D.getGraphic("deco").createBarredArea([8, 6])
// the x dimension does not matter as long as it is not 0 because
// it will be thrown away when the Form becomes "One-sided delimitation"
BarredArea.Form := "One-sided delimitation"
```

See also

Barred Area

Form [SimTalk]

`createBarredArea` [SimTalk]

[_Accessing Attributes of Graphic Shapes](#)

[_Detailed Access To Graphics](#)

[_3D.getGraphic](#) [SimTalk]

createBox [SimTalk]

Creates a box in the graphic or graphic group designated by <Path>.

Type

Method

Syntax

```
<Path>.createBox(Dimensions:length[3][, WallThickness:length]) → any
```

Parameters

You can specify the following parameters:

- The parameter *Dimensions* is an *array* of data type *length* with three values.
They designate the **Width/Dimension X**, **Depth/Dimension Y**, and **Height/Dimension Z** of the box.
- The optional parameter *WallThickness* of data type *length* sets the thickness of the walls of the box. If you do not specify the parameter, the wall thickness is 0.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Example

```
var box := _3D.getGraphic("deco").createBox([2.0,2.5,1], 0.15)
box.MaterialActive := true
box.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

See also

Box

[createRectangle \[SimTalk\]](#)

[_Detailed Access To Graphics](#)

[_3D.getGraphic \[SimTalk\]](#)

createConeFrustum [SimTalk]

Creates a cone frustum in the graphic or graphic group designated by <Path>.

Type

Method

Syntax

```
<Path>.createConeFrustum(TopRadius:length, BottomRadius:length,  
Height:length[, WallThickness:length]) → any
```

Parameters

You can specify the following parameters:

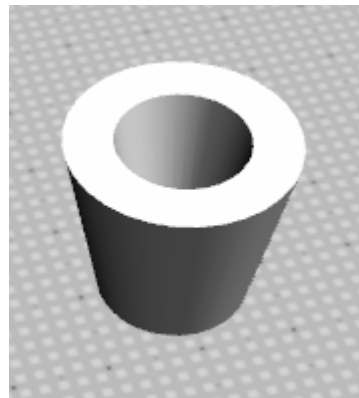
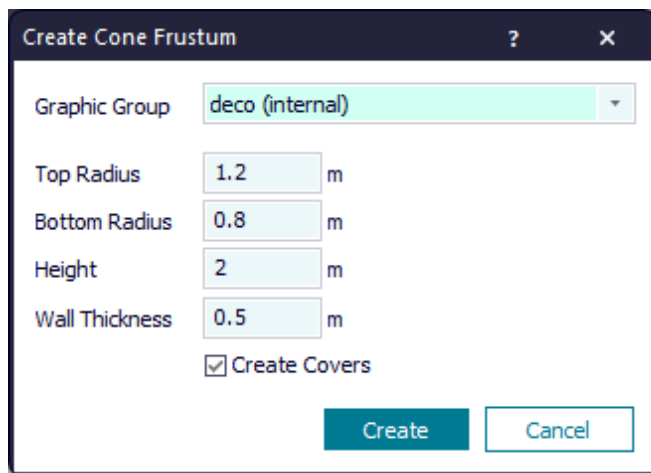
- The parameter *TopRadius* of data type *length* sets its top radius.

A **Top Radius** of 0 creates the cone with a tip.

A **Top Radius** of greater than 0 cuts off the tip of the cone. This way you can, for example, model a bottle quickly and easily.

- The parameter *BottomRadius* of data type *length* sets its bottom radius.

Type in a greater bottom radius than the top radius to create a funnel-type truncated cone.



- The parameter *Height* of data type *length* sets its height.
- The optional parameter *WallThickness* of data type *length* sets the wall thickness of the cone.

Do not specify the wall thickness to create a closed cone.

If you specify a negative value, *Plant Simulation* interprets this as if you did not specify a value.

Specify 0 to create a mantle without a thickness.

Specify a positive value to create the cone with the specified wall thickness.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Examples

```
var Cone := _3D.getGraphic("deco").createConeFrustum(2, 3, 4)
Cone.MaterialActive := true
Cone.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

```
var Cone := _3D.getGraphic("deco").createConeFrustum(2, 3, 4, 0.1)
Cone.MaterialActive := true
Cone.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

```
-- createConeFrustum, createText
if not is3DOpen
  open3D
end

var obj : object := ~.Canvas
if not obj.CreateIn3D
  obj.CreateIn3D := true
end
obj._3D.deleteGraphicGroupContent

var pos3D : length[3] := [1, 0, -0.5]
var defaultGraphicGroup : any := obj._3D.getGraphic("default")

-- bottom radius, top radius, height
-- defaultGraphicGroup.createConeFrustum(1, 0, 1.2).Position := pos3D --
German erzeugeKegel

pos3D[2] := 1.5
-- Text width 3m
defaultGraphicGroup.createText(" My text! ", 3m).Position := pos3D --
German erzeugeTextplatte
pos3D[2] := 2.5
-- defaultGraphicGroup.erzeugeText(" My large text ! ", 3m).Position :=
pos3D
```

See also

Cone Frustum

`createConeFrustum [SimTalk]`

[_Detailed Access To Graphics](#)

[_3D.getGraphic \[SimTalk\]](#)

createCuboid [SimTalk]

Creates a cuboid in the graphic or graphic group designated by <Path>.

Type

Method

Syntax

```
<Path>.createCuboid(Dimensions:length[3][, LineWidth:real,  
LineColor:integer]) → any
```

Parameters

You can specify the following parameters:

- The parameter *Dimensions* is an *array* of data type *length* with three values.
They designate the **Width/Dimension X**, **Depth/Dimension Y**, and **Height/Dimension Z** of the cuboid.
To create a flat wall, set one of the dimensions to 0. You might, for example, enter [0, 2, 3] to create a wall on the **YZ Plane** with a size of 2 times 3 meters.
- The optional parameter *LineWidth* of data type *real* designates the width of the line around the created cuboid. Enter a number greater than 0.
- The optional parameter *LineColor* of data type *integer* designates the color of the line around the created cuboid.

If you do not specify the optional parameters, the cuboid does not have a border.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Examples

```
var Cube := _3D.getGraphic("deco").createCuboid([3.0, 2.5, 1])  
Cube.MaterialActive := true  
Cube.Position := [1.0, 2.0, 0.0]  
Cube.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

```
// creates a flat wall on the YZ plane  
var Cube := _3D.getGraphic("deco").createCuboid([0, 2, 3]) // flat wall  
Cube.MaterialActive := true
```



```
Cube.Position := [1.0, 2.0, 0.0]  
Cube.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

```
-- create cuboid  
if not is3DOpen  
    open3D  
end  
  
var obj:object := ~.Canvas  
if not obj.CreateIn3D  
    obj.CreateIn3D := true  
end  
obj._3D.deleteGraphicGroupContent  
  
obj._3D.getGraphic("default").createCuboid([1.0m, 1.0m, 1.0m])
```

See also

[Cuboid](#)

[_Detailed Access To Graphics](#)

[_3D.getGraphic \[SimTalk\]](#)

createDimensioning [SimTalk]

Creates dimensionings in the graphic or graphic group designated by <Path>.

Type

Method

Syntax

```
<Path>.createDimensioning(MeasuringPoint1:length[3],  
MeasuringPoint2:length[3], TextPosition:length[3]) → any
```

Parameters

You can specify the following parameters:

- The parameter *MeasuringPoint1* is an *array* of data type *length* with three values.
They designate the **X-Position**, the **Y-Position**, and the **Z-Position** of the first measuring point.
- The parameter *MeasuringPoint2* is an *array* of data type *length* with three values.
They designate the **X-Position**, the **Y-Position**, and the **Z-Position** of the second measuring point.
- The parameter *TextPosition* is an *array* of data type *length* with three values.
They designate the **X-Position**, the **Y-Position**, and the **Z-Position** of the text that shows the measured value.

If you do not specify the parameter, *Plant Simulation* uses the center between the two measuring points as the text position.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Example

```
var Dimensioning := _3D.getGraphic("deco").createDimensioning([10,10,5],  
[5,5,5], [5,5,5])  
Dimensioning.MaterialActive := true  
Dimensioning.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

[createDimensioning \[SimTalk\]](#)

See also

[Dimensioning](#)

[DimensioningSpace \[SimTalk\]](#)

[MeasuredLength \[SimTalk\]](#)

[MeasuringPoint1 \[SimTalk\]](#)

[MeasuringPoint2 \[SimTalk\]](#)

[TextPosition \[SimTalk\]](#)

[_Detailed Access To Graphics](#)

[_3D.getGraphic \[SimTalk\]](#)

createExtrusionGraphic [SimTalk]

Creates a graphic that extrudes a two-dimensional profile along a three-dimensional vector in the graphic or graphic group designated by <Path>.

Remarks

You can create the profiles using the *F3DGenerate...* functions or use fully user-defined profiles.

A profile needs to surround an area and is not allowed to intersect itself.

Type

Method

Syntax

```
<Path>.createExtrusionGraphic(Profile:length[2,*],  
ExtrusionVector:length[3], ClosedShape:boolean) → any
```

Parameters

You can specify the following parameters:

- The parameter *Profile* is a *two-dimensional array* of data type *length* with two values. They designate the profile that will be extruded.
- The parameter *ExtrusionVector* is an *array* of data type *length* with three values. They designate the vector along which the profile will be extruded.
- The parameter *ClosedShape* of data type *boolean* specifies whether the created graphic shall be a closed shape in which case it will have a cover face at the start and at the end, or a hollow shape in which case you can look through the graphic when you look along the extrusion vector.

Note

You can create closed shapes with holes. In such a case, the shape is closed but the profile surrounds a hole, compare the examples below.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Examples

```
-- This creates a closed but hollow extruded ellipse: You can see through  
the hole but not through the graphic.  
MyObject._3D.getGraphic("default").createExtrusionGraphic(F3DgenerateHollowE  
llipse([0m, 0m], 2m, 3m, 0.1m, 0.1m), [2m, 0m, 0m], true)
```

```
-- This creates a closed cylinder: You cannot see through anything.  
MyObject._3D.getGraphic("default").createExtrusionGraphic(F3DgenerateEllips  
e([5m, 0m], 2m, 2m, 0.1m, 0.1m), [0m, 2m, 0m], true)
```

```
-- This creates the shell of a cylinder: You cannot see through anything.  
MyObject._3D.getGraphic("default").createExtrusionGraphic(F3DgenerateEllips  
e([-5m, 0m], 2m, 2m, 0.1m, 0.1m), [0m, 0m, 2m], false)
```

See also

[_Detailed Access To Graphics](#)

[createIndexedFaceSet \[SimTalk\]](#)

[_3D.getGraphic \[SimTalk\]](#)

[F3DgenerateEllipse \[SimTalk\]](#)

[F3DgenerateHollowEllipse \[SimTalk\]](#)

[F3DgenerateHollowRectangle \[SimTalk\]](#)

[F3DgenerateRectangle \[SimTalk\]](#)

[_3D.ExtConfiguration \[SimTalk\]](#)

createFactoryWalls [SimTalk]

Creates factory walls in the graphic or graphic group designated by <Path>.

Remarks

Initially the created graphic does not have any materials. You can set these later.

Type

Method

Syntax

```
<Path>.createFactoryWalls(Dimensions:length[3], WallThickness:length[,  
WallElementWidth:length]) → any
```

Parameters

You can specify the following parameters:

- The parameter *Dimensions* is an *array* of data type *length* with three values.
They designate the **Width/Dimension X**, **Depth/Dimension Y**, and **Height/Dimension Z** of the factory walls.
- The optional parameter *WallThickness* of data type *length* sets the thickness of the factory walls.
- The optional parameter *WallElementWidth* of data type *length* sets the width of the individual elements of the factory walls.

Note

When you change the setting for **Placing The Remainder**, *Plant Simulation* resets the **Cutouts**.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Example

```
var FactoryWalls := _3D.getGraphic("deco").createFactoryWalls([10, 10, 5],  
0.3)  
FactoryWalls.MaterialActive := true  
FactoryWalls.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

```
MyStation.getGraphic("default").createFactoryWalls([11,40,3], 1m)
MyStation._3D.getGraphic("default").createFactoryWalls([11,40,3],
42cm).PlaceRemainder := "Counter-clockwise"
```

See also

[Factory Walls](#)

[PlaceRemainder \[SimTalk\]](#)

[_Accessing Attributes of Graphic Shapes > _Factory Walls Attributes](#)

[_Detailed Access To Graphics](#)

[_3D.getGraphic \[SimTalk\]](#)

createFence [SimTalk]

Creates a fence in the graphic or graphic group designated by <Path>.

Remarks

Initially the created graphic does not have any materials. You can set these later.

Type

Method

Syntax

```
<Path>.createFence(Dimensions:length[3], Mesh:boolean, Panes:boolean[,  
FenceElementWidth:length]) → any
```

Parameters

You can specify the following parameters:

- The parameter *Dimensions* is an *array* of data type *length* with three values.
They designate the **Dimension X**, the **Dimension Y**, and the **Dimension Z** of the fence.
- The parameter *Mesh* of data type *boolean* sets if a wire mesh is inserted between the posts of the fence (*true*) or not (*false*).
- The parameter *Panes* of data type *boolean* sets if transparent or non-transparent panes are inserted between the posts of the fence (*true*) or not (*false*).
- The optional parameter *FenceElementWidth* of data type *length* sets the width of the individual elements of the fence.

Note

When you change the setting for **Placing The Remainder**, *Plant Simulation* resets the **Cutouts**.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Example

```
var Fence := _3D.getGraphic("deco").createFence([3.0, 2.5, 2], true, true)
Fence.MaterialActive := true
Fence.MaterialDiffuseColor := makeRGBValue(255,0,0)

MyStation.getGraphic("default").createFence([3.0, 2.5, 2], true, true, 0.5m)

MyStation1._3D.getGraphic("default").createFence([3,4,1], true,
false).PlaceRemainder := "Top and left"
```

See also

[Fence](#)

[PlaceRemainder \[SimTalk\]](#)

[_Accessing Attributes of Graphic Shapes > _Fence Attributes](#)

[_Detailed Access To Graphics](#)

[_3D.getGraphic \[SimTalk\]](#)

createIndexedFaceSet [SimTalk]

Creates an indexed face set in the graphic or graphic group designated by <Path>.

Type

Method

Syntax

```
<Path>.createIndexedFaceSet(Nodes:length[3,*], Faces:length[][][,  
Colors:integer[]]) → any
```

Parameters

You can specify the following parameters:

- The parameter *Nodes* is a *two-dimensional array* of data type *length* with three values. They designate the positions of the nodes, which follow, in space.
- The parameter *Faces* is an *array of arrays* of data type *length* which sets the number of faces which follow. Each inner *array* contains a sequence of node indexes which set the contour of the respective face.

Note

The function only works correctly, when you define convex faces.

- The optional parameter *Colors* of data type *integer* sets the color. Specify a single color to use this color for the entire graphic. Specify an *array* of colors to make each color apply to one of the faces.

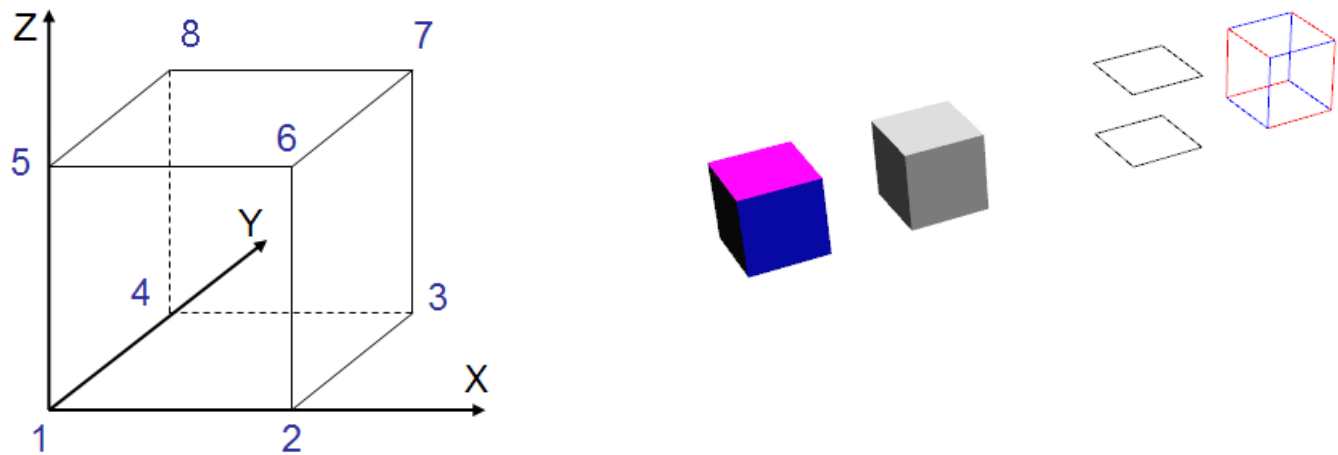
Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Explaining the Example

When we number the corner points as shown in the figure below on the left, the following example will create the graphics as shown below on the right.



Example

```

var obj := .Models.Model._3D
var Group := obj.getGraphic("deco")
for var i := Group.NumGraphics downto 1
  Group.graphic(1).deleteGraphic
next

// ##### Begin code example #####

var GraphicGroup := .Models.Model._3D.getGraphic("deco")

var Points: real[3, 8]
var b:real := 0 // bottom
var t:real := 1 // top
Points[1, 1] := b; Points[2, 1] := b; Points[3, 1] := b
Points[1, 2] := t; Points[2, 2] := b; Points[3, 2] := b
Points[1, 3] := t; Points[2, 3] := t; Points[3, 3] := b
Points[1, 4] := b; Points[2, 4] := t; Points[3, 4] := b
Points[1, 5] := b; Points[2, 5] := b; Points[3, 5] := t
Points[1, 6] := t; Points[2, 6] := b; Points[3, 6] := t
Points[1, 7] := t; Points[2, 7] := t; Points[3, 7] := t
Points[1, 8] := b; Points[2, 8] := t; Points[3, 8] := t

var Faces : any[6] := [[1,2,3,4], [5,6,7,8], [1,2,6,5], [4,3,7,8],
[2,3,7,6], [1,4,8,5]]
//
pink            blue      yellow      green
red            black
var FaceColors: integer[6] := [makeRGBValue(255,255,0),
makeRGBValue(255,0,255), makeRGBValue(0,0,255), makeRGBValue(0,255,0),
makeRGBValue(255,0,0), makeRGBValue(0,0,0)]
GraphicGroup.createIndexedFaceSet(Points, Faces, FaceColors).Position :=
[-5,0,0.5]

var FaceColor: integer := makeRGBValue(190,190,190) // grey
GraphicGroup.createIndexedFaceSet(Points, Faces, FaceColor).Position :=
[-3,0,0.5]

```

```

var PolyLine: any[2] := [[1,2,3,4,1],[5,6,7,8,5]]
GraphicGroup.createIndexedLineSet(Points, PolyLine,
makeRGBValue(0,0,0)).Position := [0,0,0.5]

var Poly3Line1: any[2] := [[1,2,6,7], [3,4,8,5]]
var Poly3Line2: any[2] := [[2,3,7,8], [4,1,5,6]]
GraphicGroup.createIndexedLineSet(Points, Poly3Line1,
makeRGBValue(255,0,0)).Position := [2,0,0.5]
GraphicGroup.createIndexedLineSet(Points, Poly3Line2,
makeRGBValue(0,0,255)).Position := [2,0,0.5]

// ##### End code example #####

```

See also

[_Detailed Access To Graphics](#)

[createIndexedLineSet \[SimTalk\]](#)

[_3D.getGraphic \[SimTalk\]](#)

createIndexedLineSet [SimTalk]

Creates an indexed line set in the graphic or graphic group designated by <Path>.

Type

Method

Syntax

```
<Path>.createIndexedLineSet(Nodes:length[3], LineSequences:integer[][][,  
Color:integer]) → any
```

Parameters

You can specify the following parameters:

- The parameter *Nodes* is a *two-dimensional array* of data type *length* with three values. They designate the positions of the nodes, which follow, in space.
- The parameter *LineSequences* is an *array of arrays* of data type *integer* which sets the sequence of lines which follow. Each inner *array* contains a sequence of node indexes which set the line.
- The optional parameter *Color* of data type *integer* sets the color.

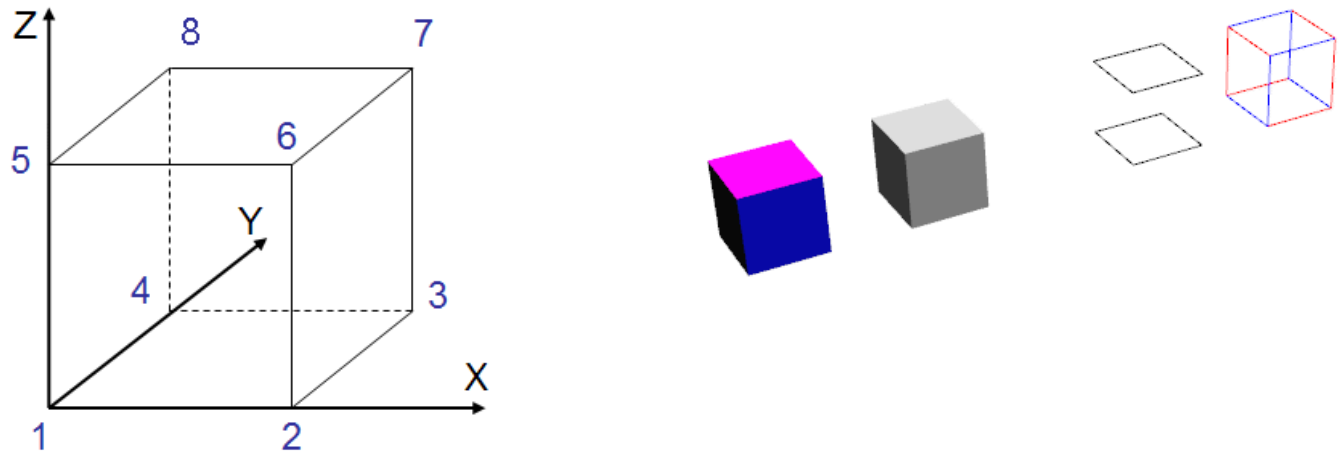
Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Explaining the Example

When we number the corner points as shown in the figure below on the left, the following example will create the graphics as shown below on the right.



Example

```

var obj := .Models.Model._3D
var Group := obj.getGraphic("deco")
for var i := Group.NumGraphics downto 1
  Group.graphic(1).deleteGraphic
next

// ##### Begin code example #####

var GraphicGroup := .Models.Model._3D.getGraphic("deco")

var Points: real[3, 8]
var b:real := 0 // bottom
var t:real := 1 // top
Points[1, 1] := b; Points[2, 1] := b; Points[3, 1] := b
Points[1, 2] := t; Points[2, 2] := b; Points[3, 2] := b
Points[1, 3] := t; Points[2, 3] := t; Points[3, 3] := b
Points[1, 4] := b; Points[2, 4] := t; Points[3, 4] := b
Points[1, 5] := b; Points[2, 5] := b; Points[3, 5] := t
Points[1, 6] := t; Points[2, 6] := b; Points[3, 6] := t
Points[1, 7] := t; Points[2, 7] := t; Points[3, 7] := t
Points[1, 8] := b; Points[2, 8] := t; Points[3, 8] := t

var Faces : any[6] := [[1,2,3,4], [5,6,7,8], [1,2,6,5], [4,3,7,8],
[2,3,7,6], [1,4,8,5]]
//
pink            blue      yellow      green
red            black
var FaceColors: integer[6] := [makeRGBValue(255,255,0),
makeRGBValue(255,0,255), makeRGBValue(0,0,255), makeRGBValue(0,255,0),
makeRGBValue(255,0,0), makeRGBValue(0,0,0)]
GraphicGroup.createIndexedFaceSet(Points, Faces, FaceColors).Position :=
[-5,0,0.5]

var FaceColor: integer := makeRGBValue(190,190,190) // grey
GraphicGroup.createIndexedFaceSet(Points, Faces, FaceColor).Position :=
[-3,0,0.5]

```

```
var PolyLine: any[2] := [[1,2,3,4,1],[5,6,7,8,5]]
GraphicGroup.createIndexedLineSet(Points, PolyLine,
makeRGBValue(0,0,0)).Position := [0,0,0.5]

var Poly3Line1: any[2] := [[1,2,6,7], [3,4,8,5]]
var Poly3Line2: any[2] := [[2,3,7,8], [4,1,5,6]]
GraphicGroup.createIndexedLineSet(Points, Poly3Line1,
makeRGBValue(255,0,0)).Position := [2,0,0.5]
GraphicGroup.createIndexedLineSet(Points, Poly3Line2,
makeRGBValue(0,0,255)).Position := [2,0,0.5]

// ##### End code example #####
```

See also

[_Detailed Access To Graphics](#)

[createIndexedFaceSet \[SimTalk\]](#)

[_3D.getGraphic \[SimTalk\]](#)

createMezzanine [SimTalk]

Creates a mezzanine in the graphic or graphic group designated by <Path>.

Remarks

Initially the created graphic does not have any materials. You can set these later.

Type

Method

Syntax

```
<Path>.createMezzanine(Dimensions:length[3], FloorThickness:length,  
PostDiameter:length, BorderLine:boolean, RailingSegmentWidth:length) → any
```

Parameters

You can specify the following parameters:

- The parameter *Dimensions* is an *array* of data type *length* with three values.
They designate the **Width/Dimension X**, the **Depth/Dimension Y**, and the **Height/Dimension Z** of the mezzanine.
- The parameter *FloorThickness* of data type *length* sets the floor thickness on which the platform of the mezzanine rests.
- The parameter *PostDiameter* of data type *length* sets the diameter of the posts on which the platform of the mezzanine rests.
- The parameter *BorderLine* of data type *boolean* sets if a borderline will be inserted around the platform of the mezzanine (*true*) or not (*false*).
- The parameter *RailingSegmentWidth* of data type *length* sets width of the individual segments of the railing.

The segment width is the distance between the individual posts that hold the railing. Type in 0 to deactivate the railing. Type in a value greater than 0 to activate the railing and to set the segment width.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Example

```
var mezzanine := _3D.getGraphic("deco").createMezzanine([1.5, 3.0, 2.8],  
0.08, 0.15, true, 1.0)  
mezzanine.MaterialActive := true  
mezzanine.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

See also

Mezzanine

[_Accessing Attributes of Graphic Shapes > _Mezzanine Attributes](#)

[_Detailed Access To Graphics](#)

[_3D.getGraphic \[SimTalk\]](#)

createPicture [SimTalk]

Creates a textured plate which shows the full-sized picture loaded from the designated file in the graphic or graphic group designated by <Path>.

Remarks

If your model contains several partially or fully transparent graphics behind each other from your current view point, it can happen that some or all of these graphics look wrong or seem to be missing.

This is due to heuristics without which the rendering performance would drastically decrease.

Type

Method

Syntax

```
<Path>.createPicture(FilePath:string, Orientation:integer, Size:length[2],  
Bifacial:boolean[, Thickness:length:=1mm, UseTransparency:boolean:=true])  
→ any
```

Parameters

You can specify the following parameters:

- The parameter *FilePath* of data type *string* sets the path and the name of the file you want to import.
- The parameter *Orientation* of data type *integer* sets the sets the orientation of the picture.
1 stands for **Floor**, 2 stands for **Front Wall**, and 3 stands for **Side Wall**.
- The parameter *Size* is an *array* of data type *length* with two values.
They designate the **Width** and the **Height** of the picture. If you enter a value less than or equal to 0 for one of the values for one of the two values, the respective other value is computed from the width to height ratio.
- The parameter *Bifacial* of data type *boolean* sets if the texture will be shown on both sides of the plate (*true*) or only on the top side of the plate (*false*).
- The optional parameter *Thickness* of data type *length* sets the thickness of the plate.

Default Value of the Parameter

The default value is 1 mm.

- The optional parameter *UseTransparency* of data type *boolean* sets if transparent or partially transparent pixels in the texture are shown transparent (*true*) or if *Plant Simulation* enforces opacity (*false*).

If you don't specify this parameter, *Plant Simulation* will use transparency.

If the image file does not contain transparent pixels, the parameter has no effect.

Default Value of the Parameter

The default value is true.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Example

```
var picture := _3D.getGraphic("deco").createPicture("D:\Picture.png", 1,  
[5.0,3.0], false, 1)  
picture.MaterialActive := true  
picture.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

See also

[Textured Plate](#)

[createTiledPlate \[SimTalk\]](#)

[_3D.getGraphic \[SimTalk\]](#)

[_Detailed Access To Graphics](#)

createPolyLine [SimTalk]

Creates a polyline with the specified parameters in your 3D simulation model specified by <Path>.

Type

Method

Syntax

```
<Path>.createPolyLine(Nodes:length[3,*], Diameter:length [,
Directed:boolean:=false]) → any
```

Parameters

You can specify the following parameters:

- The parameter *Nodes* is an *array* of data type *length*.
They designate the anchor points of the polyline.
- The parameter *Diameter* of data type *length* designates the diameter of the polyline.
- The optional parameter *Directed* of data type *boolean* sets if the direction of the polyline will be shown (*true*) or not (*false*).

Default Value of the Parameter

The default values is *false*.

Return Value

The return value has the data type *any*.

It designates the created polyline, compare the method `_3D.getGraphic`.

Example

```
// The following code creates a directed polyline with 4 points, a
// diameter of 0.1 and a color of orange.
var Points = make2DimArray(3, [ 6.0, -2.0, 1.0,
                                9.0, -2.0, 1.0,
                                12.0, -4.0, 2.0,
                                15.0, -4.0, 2.0])

var PolyLine := _3D.getGraphic("deco").createPolyLine(Points, 0.1, true)
PolyLine.MaterialDiffuseColor := makeRGBValue(255, 128, 0)
```

See also

[_Detailed Access To Graphics](#)

[_3D.getGraphic \[SimTalk\]](#)

createRack [SimTalk]

Creates a rack in the graphic or graphic group designated by <Path>.

Remarks

Initially the created graphic does not have any materials. You can set these later.

Type

Method

Syntax

```
<Path>.createRack(BaySize:length[2], BoardDepth:length,
Capacity:integer[2], MinimalStructure:boolean[, SquarePosts:boolean:=false,
PostDiameter:length:=40mm, BoardThickness:length:=30mm,
GroundClearance:length]) → any
```

Parameters

You can specify the following parameters:

- The parameter *BaySize* is an *array* of data type *length* with two values.
They designate the **Width/Dimension X** and the **Height/Dimension Z** of the individual bays of the rack.
- The parameter *BoardDepth* of data type *length* sets the **depth/Dimension Y** of the boards of the rack.
- The parameter *Capacity* is an *array* of data type *integer* with two values.
They designate the capacity of the rack, i. e., the **Number of Bays per Board** and the **Number of Boards**.
- The parameter *MinimalStructure* of data type *boolean* sets if the optimization is aiming for **Performance** (*true*) or for **Post-processing Capability** (*false*).
- The optional parameter *SquarePosts* of data type *boolean* sets if the rack will be created with square posts (*true*) or with round posts (*false*). If you do not specify the parameter, the rack will be created with round posts.
- The optional parameter *PostDiameter* of data type *length* sets the diameter of the posts of the rack.

Default Value of the Parameter

The default value is 4 cm.

- The optional parameter *BoardThickness* of data type *length* sets the thickness of a board of the rack.

Default Value of the Parameter

The default value is 3 cm.

- The optional parameter *GroundClearance* of data type *length* sets the distance between the lower edge of the lowest board and the floor on which the rack sits. Type in 0 so that the rack sits flat on the floor.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Example

```
var Rack := _3D.getGraphic("deco").createRack([1.5, 1.5], 1, [3, 4], true,
true, 0.1, 0.05, 0.5))
Rack.MaterialActive := true
Rack.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

See also

[Rack](#)

[_Accessing Attributes of Graphic Shapes > _Rack Attributes](#)

[_Detailed Access To Graphics](#)

[_3D.getGraphic \[SimTalk\]](#)

createRectangle [SimTalk]

Creates a rectangle in the graphic or graphic group designated by <Path> at the position (X,Y) with the specified **Width** and **Height**.

Remarks

If you specify a negative **Width**, **X** designates the **X-Position** of the right hand side.

If you specify a positive **Height**, **Y** designates the **Y-Position** of the bottom side.

Type

Method

Syntax

```
<Path>.createRectangle(X:real, Y:real, Width:real, Height:real[,  
Color:integer=-1, Thickness:real=0.08]) -> any
```

Parameters

You can specify the following parameters:

- The parameter *X* of data type *real* sets the **X-position** at which the rectangle is going to be created in the window.
- The parameter *Y* of data type *real* sets the **Y-position** at which the rectangle is going to be created in the window.
- The optional parameter *Color* of data type *integer* sets the **Color** of the rectangle.

Default Value of the Parameter

The default value is -1.

- The optional parameter *Thickness* of data type *real* sets the **Thickness** of line that forms the rectangle.

Default Value of the Parameter

The default value is 0.08.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Example

```
-- Creates a rectangle in the graphic group named deco
-- with the top left corner located at the position (2,-2),
-- a width of 8, and a height of 6. The color is the default color 1 and
-- the line width is 0.1.
root._3D.deco.createRectangle(2, -2, 8, -6, getStandardColor(1), 0.1)
```

```
-- Creates a rectangle in the graphic group named deco
-- with the bottom left corner located at the position (2,-2),
-- a width of 8, and a height of 6. The color is the default color 1 and
-- the line width is 0.1.
root._3D.deco.createRectangle(2, -2, 8, 6, getStandardColor(1), 0.1)
```

```
-- Creates a rectangle in the graphic group named deco
-- with the lower left corner located at the position (2,-2),
-- a width of 8, and a height of 6. The color is RGB(200, 0, 0) and
-- the line width is the default line width of 0.08.
root._3D.getGraphic("deco").createRectangle(2, -2, -8, 6, makeRGBValue(200,
0, 0))
```

See also

[Box](#)

[createBox \[SimTalk\]](#)

[_Detailed Access To Graphics](#)

[_3D.getGraphic \[SimTalk\]](#)

createSphere [SimTalk]

Creates a sphere in the graphic or graphic group designated by <Path>.

Type

Method

Syntax

```
<Path>.createSphere(Radius:length) → any
```

Parameter

The parameter *Radius* of data type *length* sets the radius of the sphere.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Example

```
var Sphere := _3D.getGraphic("deco").createSphere(2)
Sphere.MaterialActive := true
Sphere.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

See also

[Sphere](#)

[_Detailed Access To Graphics](#)

[_3D.getGraphic \[SimTalk\]](#)

createStairs [SimTalk]

Creates a staircase in the graphic or graphic group designated by <Path>.

Remarks

Initially the created graphic does not have any materials. You can set these later.

Type

Method

Syntax

```
<Path>.createStairs(TreadWidth:length, Rise:length,  
StartAtFirstTread:boolean) → any
```

Parameters

You can specify the following parameters:

- The parameter *TreadWidth* of data type *length* sets the tread width. This is the width of the steps and thus the width of the entire stairway.
- The parameter *Rise* of data type *length* sets the rise. This is the height of the entire stairway from the floor of one floor to the ceiling of the next floor.
- The parameter *StartAtFirstTread* of data type *length* sets if the staircase starts at the first tread without the stringer jutting out onto the floor on which the stairway rests (*true*) or not (*false*).

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Example

```
var Stairs := _3D.getGraphic("deco").createStairs(1.5, 3, true)  
Stairs.MaterialActive := true  
Stairs.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

See also

Stairs

[_Accessing Attributes of Graphic Shapes](#) > [_Stairs Attributes](#)

[_Detailed Access To Graphics](#)

[_3D.getGraphic \[SimTalk\]](#)

createText [SimTalk]

Creates text in the graphic or graphic group designated by <Path>.

Type

Method

Syntax

```
<Path>.createText(Text:string[, Width:length, TextColor:integer,  
BackgroundColor:integer, Alignment:integer]) → any
```

Parameters

You can specify the following parameters:

- The parameter *Text* of data type *string* sets the actual text you want to show.
- The optional parameter *Width* of data type *length* sets the width of the text.
To automatically compute the width according to the length of the text, enter -1.
- The optional parameter *TextColor* of data type *integer* sets the color of the text.
- The optional parameter *BackgroundColor* of data type *integer* sets the background color of the text.
- The optional parameter *Alignment* of data type *integer* sets the alignment of the text.
-1 aligns the text at the **left**, 0 **centers** the text, 1 aligns it at the **right**.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method `_3D.getGraphic`.

Example

```
var Text := _3D.getGraphic("deco").createText("My text.", 6) // width 6m  
Text.MaterialActive := true  
Text.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

See also

[Text \[graphic shape\]](#)

[_3D.getGraphic \[SimTalk\]](#)

Detailed Access To Graphics

createTiledPlate [SimTalk]

Creates a plate which shows the picture or the texture loaded from a picture tiled in the graphic or graphic group designated by <Path>.

Remarks

If your model contains several partially or fully transparent graphics behind each other from your current view point, it can happen that some or all of these graphics look wrong or seem to be missing.

This is due to heuristics without which the rendering performance would drastically decrease.

Type

Method

Syntax

```
<Path>.createTiledPlate(FilePath:string, Orientation:integer,
Size:length[2], Bifacial:boolean, TileSize:length[2],
Thickness:length:=1mm, UseTransparency:boolean:=true) → any
```

Parameters

You can specify the following parameters:

- The parameter *FilePath* of data type *string* sets the path and the name of the file you want to import.
- The parameter *Orientation* of data type *integer* sets the sets the orientation of the picture.
1 stands for **Floor**, 2 stands for **Front Wall**, and 3 stands for **Side Wall**.
- The parameter *Size* is an *array* of data type *length* with two values.
They designate the **Width** and the **Height** of the picture. If you enter a value less than or equal to 0 for one of the values for one of the two values, the respective other value is calculated from the width to height ratio. You can enter a maximum **Width** or **Height** of 32767 pixels.
- The parameter *Bifacial* of data type *boolean* sets if the texture will be shown on both sides of the plate (*true*) or only on the top side of the plate (*false*).
- The parameter *TileSize* is an *array* of data type *length* with two values.
They designate the **Width/Tile Size X** and the **Depth/Tile Size Y** of a tile. If you enter a value less than or equal to 0 for one of the values for one of the two values, the respective other value is calculated from the width to height ratio.
- The optional parameter *Thickness* of data type *length* designates the thickness of the plate.

Default Value of the Parameter

The default value is 1 mm.

- The optional parameter *UseTransparency* of data type *boolean* sets if transparent or partially transparent pixels in the texture are shown transparent (*true*) or if *Plant Simulation* enforces opacity (*false*). If you don't specify this parameter, *Plant Simulation* will use transparency.

If the image file does not contain transparent pixels, the parameter has no effect.

Default Value of the Parameter

The default value is *true*.

Return Value

The return value has the data type *any*.

It designates the created graphic, compare the method *_3D.getGraphic*.

Example

```
var TiledPlate := _3D.getGraphic("deco").createTiledPlate("D:\Picture.png",
1, [4.0,2.0], true, [2.0, 2.0], 0.1)
TiledPlate.MaterialActive := true
TiledPlate.MaterialDiffuseColor := makeRGBValue(255,0,0)
```

See also

[Textured Plate](#)

[createPicture \[SimTalk\]](#)

[_3D.getGraphic \[SimTalk\]](#)

importGraphics [SimTalk]

Imports the specified graphic from the file and adds it to the existing graphic in the graphic or graphic group designated by <Path>.

Remarks

Plant Simulation can import files of these types:

```
All 3D Graphic Files (*.dgn;*.dwg;*.dxf;*.exp;*.iges;*.igs;*.jt;*.model;*.step;*.stl;*.stp;*.vrm;*.wrl;*.x_b;*.x_t;*.xmt_bin;*.xmt_txt)
JT Files (*.jt)
Parasolid Files (*.x_b; *.x_t; *.xmt_bin; *.xmt_txt)
IGES Files (*.iges; *.igs)
STEP Files (*.step; *.stp)
VRML Files (*.vrm; *.wrl)
STL Files (*.stl)
CATIA V4 Files (*.exp; *.model)
CAD Layout Files (*.dgn; *.dwg; *.dxf)
```

Note

Plant Simulation uses the file name of the graphic as the name of the graphic node.

Type

Method

Syntax

```
<Path>.importGraphics(FilePath:string[, YUpToZUp:boolean:=false,
MoveContentToZero:boolean:=false]) → any
```

Parameters

You can specify the following parameters:

- The parameter *FilePath* of data type *string* sets path to the graphic to be imported.

Default Value of the Parameter

The default value is false.

- The optional parameter *YUpToZUp* of data type *boolean* sets if up-direction, i. e., the y-direction of the imported graphic, will be converted from the y-direction to the z-direction (*true*) or not (*false*).
- The optional parameter *MoveContentToZero* of data type *boolean* sets if the visible center of the imported graphic is to be moved to the center of the scene defined by the coordinates 0,0,0 (*true*) or not (*false*).